UNIVERSITY OF PRETORIA
DEPARTMENT OF COMPUTER SCIENCE
COS 214 – SOFTWARE MODELLING PROJECT

STUDENT INFORMATION	
NAME AND SURNAME	**STUDENT NUMBER**
MOGALE LEBETHE	25237269
CHAITANYA SUBBIAH	24744019
YARIV SINGH	24612082
LEENSA KETEREW	24577988
JOELLE TSAKU	24984362
CHELSY WHATA	25657764
TERRENCE HLUYO	25446968

Table of Contents

Research on Workflow Management Systems 3
Define the Scenario and System 4
UML Activity Diagrams 6
- Activity Diagram 1 (Generic Workflow) 6
- Activity Diagram 2 (Generic Approval and Escalation) 7
- Activity Diagram 3 (Agency Client Lifecycle) 9
- Activity Diagram 3b (Campaign Production) 12

Design Patterns applied in the TIVIDY architecture 13
- Composite 13
- Prototype 14
- Builder 14
- State 15
- Flyweight 16
- Command 16
- Memento 17
- Observer 18
- Strategy 18
- Facade 19

- Runtime Behaviour Diagrams: State Diagram (Task Lifecycle) 20

Runtime Behaviour Diagrams 22

Runtime Behaviour Diagrams: State Diagram (Task Lifecycle)....22
Design Decisions and Revisions....23
Revision 1: From the whiteboard plan to a generic workflow engine....23
Revision 2: From a fixed engine design to the implemented design....23
GitHub Workflow....25

Research on Workflow Management Systems

A workflow management system (WfMS) is an information system used by businesses designed to automatically assign tasks to resources—humans and software applications—according to a predefined schema of process rules, resources capabilities, and dependencies [1]. The main objective of a WfMS is to separate process coordination from application-specific execution logic. While traditional software models hardcoded business processes directly into application code, modern architectures split systems into a logistical management subsystem (the workflow engine) and task-execution applications [2]. The logistical subsystem coordinates case completion, enforces activity sequencing, manages parallel execution, and handles resource assignment without altering case payload data directly [2].

A fundamental architectural requirement of any generic WfMS is the strict operational decoupling between a *workflow definition* and a *running instance* [2], [3]. A workflow definition acts as a blueprint specifying the structural model, activity sequences, routing rules, required agent roles, and transition conditions [2], [3]. Conversely, a running workflow instance represents an active execution thread instantiated from that definition [3]. The definition contains schema-level invariants, whereas the instance encapsulates runtime state, execution data attributes, active worklists, audit logs, and assigned participants [2], [3], [4]. This separation ensures that multiple workflow instances can execute concurrently without changing the underlying process model.

Within a WfMS, work is modelled abstractions termed *activities* or *work items* [3], [5]. Complex nested workflows and sub-processes are typically organised using hierarchical objects structures or dynamic event arrays [3], [4]. For instance, systems such as YAWL utilise “worklets”—extensible repertoires of self-contained sub-processes selected at runtime—to execute nested tasks.

Work items are assigned and routed dynamically using configurable strategies rather than hard-coded logic [3], [4]. To prevent rigid coupling between specific individuals and task definitions, participant models integrate *role-based abstractions* and object evolution mechanisms [3]. Under a role model, human workers and automated software agents acquire and relinquish roles dynamically [4]. Agent selection policies and routing rules evaluate worker availability, workload distributions, and process execution history [2], [4]. Worklist management policies monitor temporal constraints by rescheduling pending activities as new work arrives, or by triggering reminders when deadlines approach [4].

To achieve adaptability to changing operational requirements, object-oriented architectures use *Event/Condition/Action (ECA) rules*, which encapsulate activity ordering (sequencing, branching, and joining), dependency resolution, and agent selection into customisable objects [4]. Dependencies between steps are enforced through transition guards and signal evaluation [2], [4]. The workflow management level interacts with external systems by passing case data attributes upon task startup and receiving updated attributed upon task completion [2] . For interoperability with legacy systems, component-oriented adapters are typically used [5].

Workflow systems handle mid-execution modifications through *dynamic workflows* and *exception handling* [3]. Dynamic workflow adaptation addresses process model evolutions by allowing running instances to co-exist across multiple versions, typically by enabling authorised ad-hoc modifications (e.g., skipping, or reordering tasks) [3]. Exception handling isolates deviations (e.g., resource unavailability, or system failures) from the main process model [3]. Instead of cluttering the primary process flow with exhaustive error branches, decoupled exception handlers or rule-driven sub-processes are dynamically dispatched to resolve issues, log audit histories, execute rollbacks, or escalate cases to administrators without causing system downtime [3], [4].

Define the Scenario and System

Initial scope of TIVIDY

Main responsibilities: Workflows are created instances derived from definitions, TIVIDY assigns work to participants by role, it tracks each task through its lifecycle, handles approvals and escalation, reacts to events and keeps a full history.

Boundaries: TVIDY will manage the process only. It does not do the work itself and does not own the external systems. Payments, Email, e-signature and ad platforms are reached through adapters and will be simulated.

Definition vs instance: WorkflowDefinition holds the stages, roles, tasks, approvals chain and conditions it is also reused. WorkflowInstance holds the running state, as well as task states, WorkflowData e.g. discount, the assigned participants and the history.

Variance Expected: How work is assigned, levels of approval, external systems, additional teasks needed, the reactor to events and required reports. Engine classes will remain.

Selected Scenario: Client Lifecycle at an Advertising Agency

Organisation and environment

- Kinetic Media is a fictional mid-sized agency in Pretoria that sells monthly campaign packages (Bronze, Silver or Gold, plus add-ons) to small businesses. Today it works through email and spreadsheets, so leads are forgotten and discounts go out without sign-off.

Process being managed

- The Agency Client Lifecycle runs from first contact to the end of the contract. It has six stages: acquisition, brief and proposal, onboarding, campaign production, launch and monitoring, and cancellation and win-back.

Participants and roles

- Sales Agent, Account Manager, Account Director, Managing Director, Copywriter, Designer, Media Planner, Creative Director, Compliance Officer, Finance and Client.

Major pieces of work

- Contact the lead, gather requirements, build the proposal, approve the discount, send the contract and issue the deposit invoice and create the account in parallel, write the brief, produce the copy, visuals and media plan in parallel, run the three reviews, launch, send the monthly report, and send the win-back offer.

Decisions and routing points

- Is the lead interested, is a discount requested and approved, does the client accept, request changes or reject (at most 3 rounds), were the contract signed and deposit paid within 7 days, did the reviews pass, has the term ended, did the client cancel, and was the win-back accepted within 14 days.

Approvals and dependencies

- Discounts go to the Account Manager up to 10%, the Account Director up to 25% and the Managing Director above that, and an approver who does not respond in 2 days escalates to the next level.
- The proposal needs an approved discount. The team needs both a signed contract and a paid deposit. Internal review needs all three creative pieces. Launch needs a passed compliance check.

Failures and exceptions

- The lead does not respond, an approver or client does not respond in time, the contract or deposit is late, an external call fails (retry 3 times, then restore the snapshot and mark the instance Failed), a review rejects the work (reopen the task), or the client cancels (snapshot and win-back).

External systems

- An email server (LegacySmtpClient through EmailAdapter), an SMS gateway (SmsGatewayApi through SmsAdapter), an e-signature service (ESignService through ESignConnector), a payment gateway (PaymentGatewayApi through PaymentConnector) and an ad platform (AdPlatformApi through AdPlatformConnector).

Expected outcome

Every workflow ends closed as Completed, Lost, Churned or Failed. Nothing is deleted, and the full history is kept for audit

UML Activity Diagrams

Activity Diagram 1 (Generic Workflow)

This diagram shows how TIVIDY runs any workflow instance, from the moment a user requests one until the request ends. It is generic and does not depend on any scenario. It has four swimlanes: User / TextUI, WorkflowEngineFacade / Registries, WorkflowFactory / Workflow model, and ProjectSession / CommandHistory.

The flow starts when the user asks for a workflow instance or a project clone. The Facade receives either startInstance(definitionId, instanceName) or cloneProject(projectId, newName), and the creation path depends on the request:

- **Definition:** DefinitionRepository::find() locates the definition. WorkflowFactory::instantiate() validates the template and builds it with the Director and the builders.
- **Existing project:** ProjectRegistry::findProject() finds the project. WorkflowFactory::duplicate() calls Project::clone(), which resets the runtime data and remaps the dependency edges.

If creation fails, a WorkflowError is shown and the attempt ends. Otherwise ProjectRegistry::add(project) creates a ProjectSession with its own CommandHistory and AuditLog.

The user then enters the main loop. The Facade returns the available work and the project status, which ReportService reads from the project tree. The user chooses the next request, and the Facade resolves the project and the target task:

- performAction(), decide(), editTask(), addTaskToProject() or addDependency() create a new command, which the project's CommandHistory runs.
- undo() and redo() delegate to the project's CommandHistory. The Facade owns no undo or redo stacks.

If the project or task cannot be found, a lookup error is shown and no command runs. If undo or redo is requested while the required stack is empty, a WorkflowError is thrown and nothing changes. Otherwise:

- **Run or redo:** Command::execute() applies the operation.
- **Undo:** Command::undo() restores the recorded state, attributes, selection or structure.

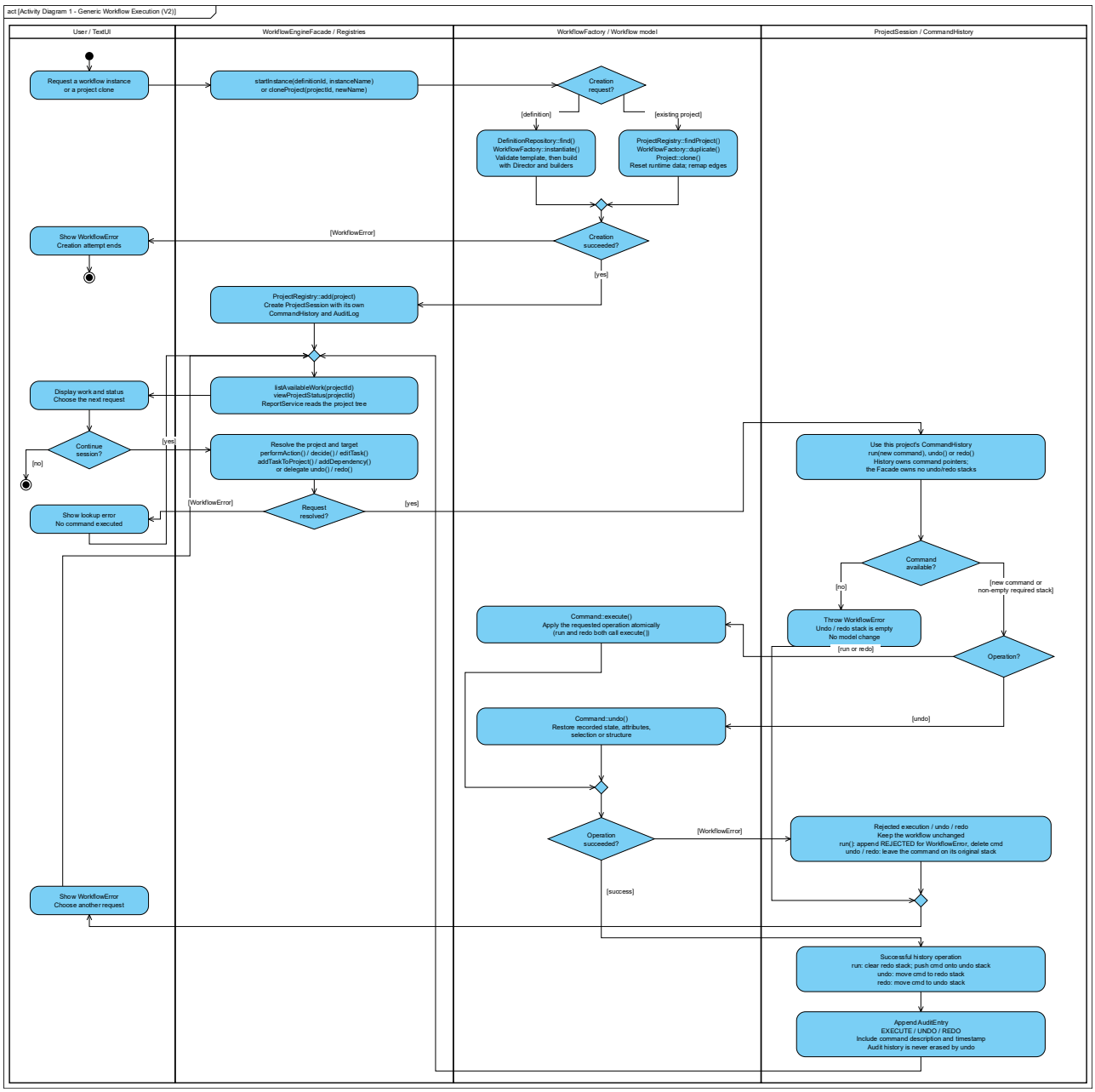
If the operation fails with a WorkflowError, the workflow stays unchanged:

- **Run:** an entry marked REJECTED is appended to the audit log and the command is deleted.
- **Undo or redo:** the command stays on its original stack.

If the operation succeeds, CommandHistory updates its stacks:

- **Run:** the redo stack is cleared and the command is pushed onto the undo stack.
- **Undo:** the command moves to the redo stack.
- **Redo:** the command moves back to the undo stack.

An AuditEntry (EXECUTE, UNDO or REDO, with the command description and a timestamp) is appended. Undo never erases the audit history. The loop repeats until the user ends the session.



Activity Diagram 2 (Generic Approval and Escalation)

This diagram shows what happens when a task changes state, fails, and is then recovered. It has five swimlanes: User, Dependent components (Observer), WorkflowEngineFacade (Facade + ActionRegistry), ProjectSession + CommandHistory + AuditLog (Command), and Task + TaskState (State / Flyweight).

The user chooses an action on a task (start, complete, fail, block, cancel and so on). performAction(projectId, taskId, action) finds the task, and ActionRegistry::create() builds the matching Command. ProjectSession::executeCommand() passes it to CommandHistory, which calls execute(). The task passes the event to its current TaskState object, a shared flyweight. If the transition is not allowed, an InvalidTransitionError is thrown. The command is recorded as REJECTED and deleted, the user sees the error, and nothing changes or is added to the undo stack.

If the transition is allowed, the task calls setState(next state) and notifyDependents(). Each dependent's update() recomputes its readiness:

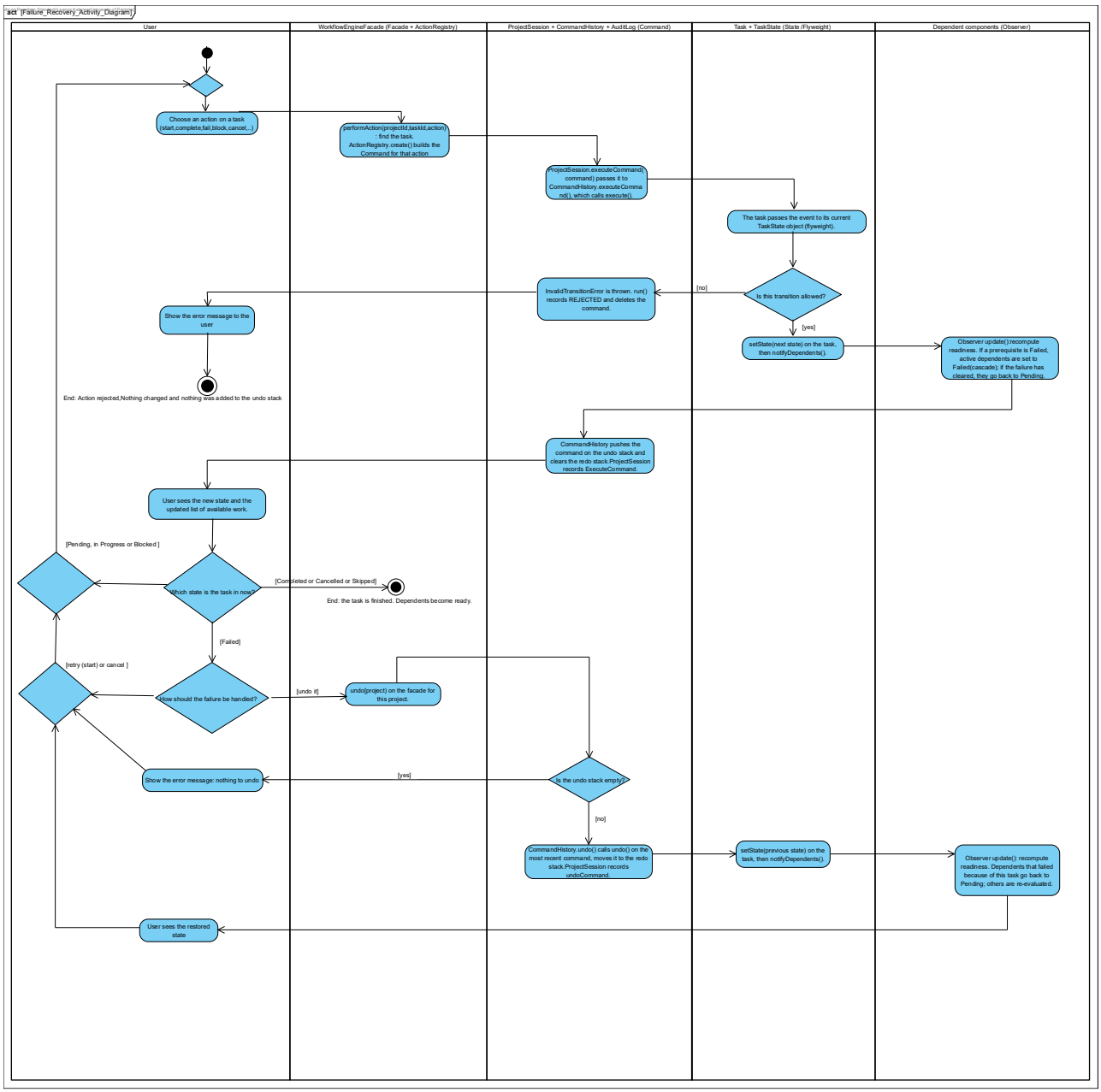
- **Prerequisite Failed:** active dependents are set to Failed (cascade).
- **Failure cleared:** dependents that failed because of it go back to Pending.

The command is pushed onto the undo stack, the redo stack is cleared, and ExecuteCommand is recorded. The user sees the new state and the updated list of available work.

The diagram then asks which state the task is in:

- **Pending, InProgress or Blocked:** the task continues normally.
- **Complete, Cancelled or Skipped:** the task is finished and its dependents become ready.
- **Failed:** the user chooses how to handle it. They can retry (start) or cancel, or they can undo the command.

To undo, the user calls undo(project) on the Facade. If the undo stack is empty, an error says there is nothing to undo. Otherwise CommandHistory::undo() calls undo() on the most recent command and moves it to the redo stack, and the session records UndoCommand. The task goes back to its previous state with setState(previous state) and notifyDependents(), and the dependents re-evaluate. Any dependent that failed because of this task returns to Pending. The user sees the restored state.



Activity Diagram 3 (Agency Client Lifecycle)

The scenario is one WorkflowDefinition running on TIVIDY. The swimlanes are the Sales Agent, the Account Manager, the Client and Agency Operations / TIVIDY.

The flow starts when a new lead is recorded. The sales agent contacts the client and records the attempt. If there is no response and fewer than 3 attempts have been made, the contact is retried. If the client is not interested, the lead outcome is recorded and the enquiry is closed as unsuccessful. Otherwise the requirements are recorded and an account manager is assigned.

The account manager builds a base package with add-ons. If a discount is required, the Approve Discount sub-activity runs. If the discount is rejected or the package needs changes, the package is rebuilt. Otherwise the proposal is prepared and sent. The client reviews it and has three outcomes:

- **Refuse:** the refusal is recorded and the enquiry is closed as unsuccessful.
- **Request a revision:** the package is rebuilt.
- **Proceed:** the flow moves to onboarding.

During onboarding, three things happen in parallel (fork/join): the signed contract, the payment confirmation and the client assets are recorded. A 7-day onboarding check follows. If onboarding is not complete, the failure is recorded, the remaining onboarding work is cancelled and the flow ends. Otherwise executors and deadlines are assigned and Campaign Production (Activity Diagram 3b) runs.

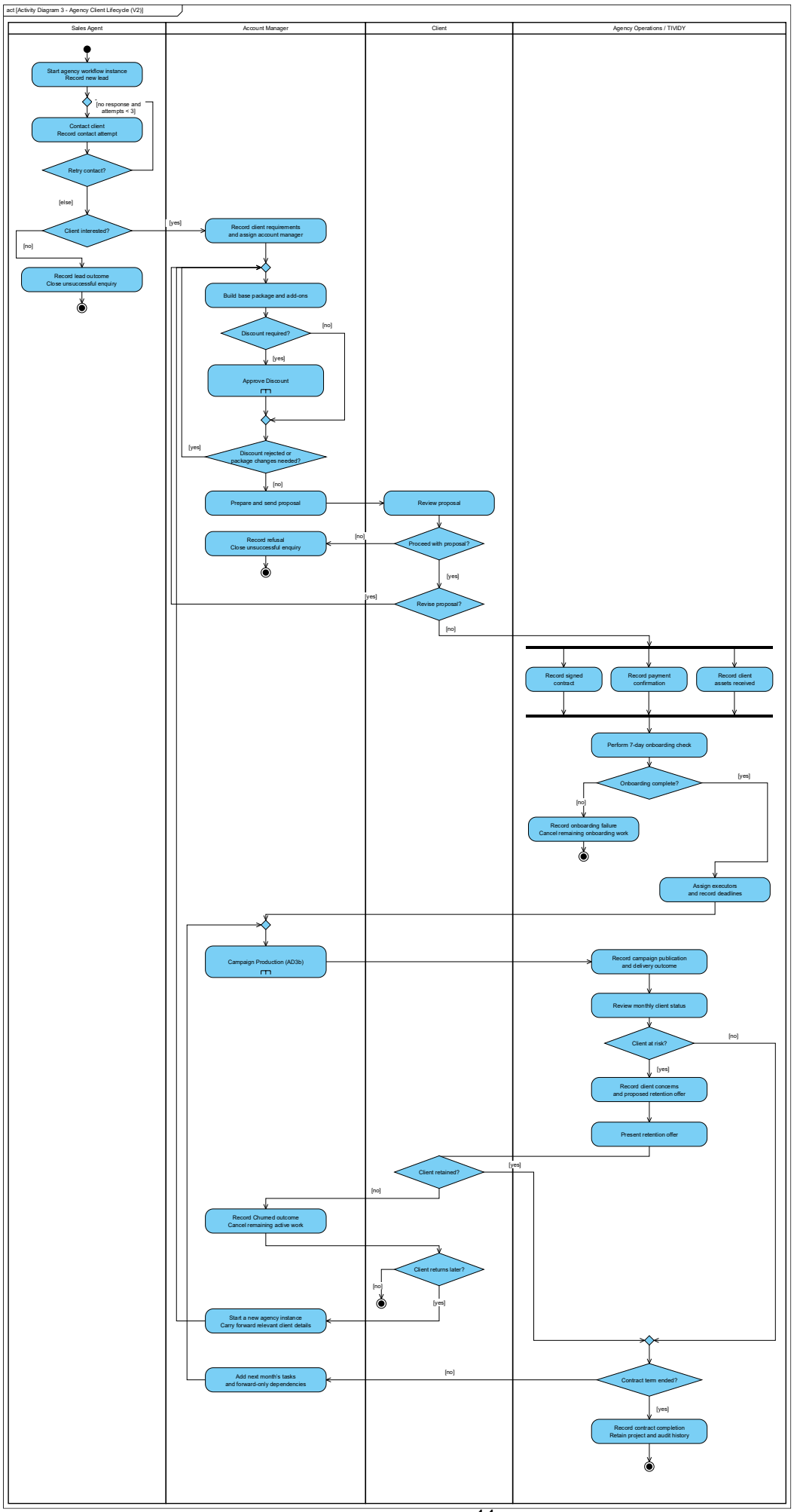
Each month the campaign publication and delivery outcome are recorded and the client status is reviewed. If the client is at risk, the client's concerns and a retention offer are recorded and the offer is presented:

- **Client retained:** the monthly cycle continues.
- **Client not retained:** the Churned outcome is recorded and the remaining active work is cancelled. If the client returns later, a new agency instance starts and carries forward the relevant client details.

If the contract term has not ended, the account manager adds next month's tasks and forward-only dependencies, and Campaign Production runs again. When the term has ended, the contract completion is recorded and the project and its audit history are retained. Nothing is deleted.

Traceability to the class design

- Lanes (Sales Agent, Account Manager, Client…): the executor of a task, set through Participant (id, name, role) with setExecutor().
- Every box: a Task inside the Project (Composite). Its lifecycle comes from TaskState (State).
- Lead interested? / Discount required? / Proceed with proposal?: a DecisionTask. A RoutingStrategy chooses the branch from the JSON context, and non-selected branches are Skipped.
- Contact retry and monthly cycle: new tasks and forward-only dependencies added with AddWorkItemCommand and AddDependencyCommand. Dependencies never form a cycle (CyclicDependencyError).
- Onboarding fork / join: three Tasks with no dependencies between them. A fourth Task depends on all three and becomes ready only when they are Complete (Observer).
- Approve Discount: an industry action, approve or reject, registered in the ActionRegistry (Command).
- Close as Lost / Churned / Completed: tasks moved to Cancelled or Complete. The AuditLog keeps the history.
- Start an instance from the definition: WorkflowFactory::instantiate() (Builder).
- Carry forward client details on a return: cloneProject() (Prototype).


act [Activity Diagram 3 - Agency Client Lifecycle (V2)]
Sales Agent
Account Manager
Client
Agency Operations / TIVIDY
Start agency workflow instance
Record new lead
[no response and attempts < 3]
Contact client
Record contact attempt
Retry contact?
[else]
Client interested?
[yes]
[no]
Record client requirements and assign account manager
Record lead outcome
Close unsuccessful enquiry
Build base package and add-ons
Discount required?
[no]
[yes]
Approve Discount
[yes]
Discount rejected or package changes needed?
[no]
Prepare and send proposal
Review proposal
Record refusal
Close unsuccessful enquiry
[no]
Proceed with proposal?
[yes]
[yes]
Revise proposal?
[no]
Record signed contract
Record payment confirmation
Record client assets received
Perform 7-day onboarding check
Onboarding complete?
[yes]
[no]
Record onboarding failure
Cancel remaining onboarding work
Assign executors and record deadlines
Campaign Production (AD3b)
Record campaign publication and delivery outcome
Review monthly client status
Client at risk?
[no]
[yes]
Record client concerns and proposed retention offer
Present retention offer
Client retained?
[yes]
[no]
Record Churned outcome
Cancel remaining active work
Client returns later?
[no]
[yes]
Start a new agency instance
Carry forward relevant client details
Add next month's tasks and forward-only dependencies
[no]
Contract term ended?
[yes]
Record contract completion
Retain project and audit history

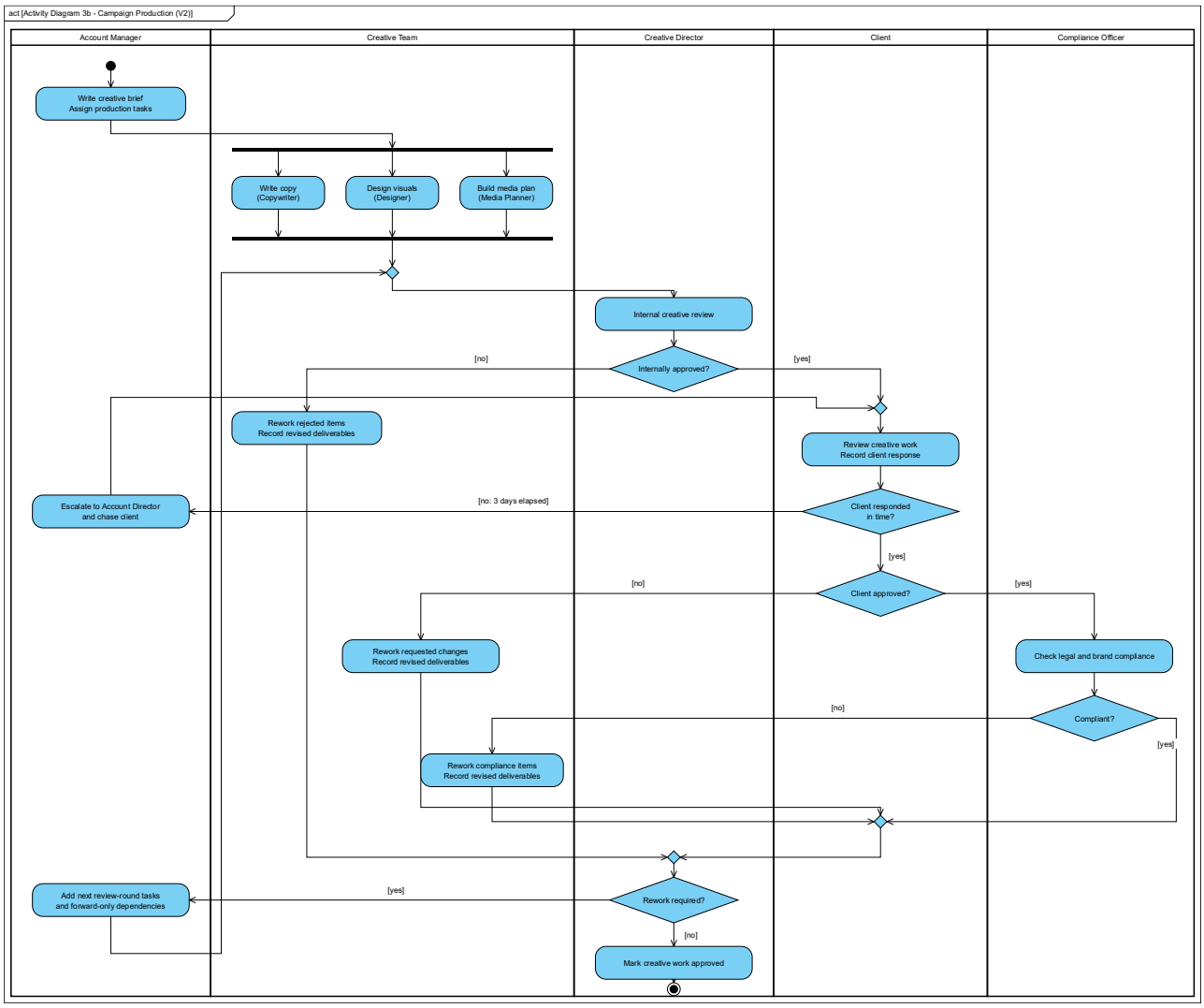
Activity Diagram 3b (Campaign Production)

This is the composite activity inside Activity Diagram 3. The diagram has five swimlanes: Account Manager, Creative Team, Creative Director, Client and Compliance Officer.

The account manager writes the creative brief and assigns the production tasks. Copywriting, visual design and the media plan are produced in parallel (fork/join). The work then goes through three reviews:

1. **Internal creative review** by the Creative Director. If it is not approved, the rejected items are reworked and the revised deliverables are recorded.
2. **Client review.** If the client has not responded within 3 days, the account manager escalates to the Account Director and chases the client. If the client is not satisfied, the requested changes are reworked.
3. **Compliance check** for legal and brand compliance. If the work is not compliant, the compliance items are reworked.

All rework joins at one merge point. If rework is required, the account manager adds the next review-round tasks and forward-only dependencies, and the reviews run again. When all three reviews pass, the creative work is marked approved and the activity ends.



Traceability (AD3b)

- Copy, visuals and media plan in parallel: three Tasks with no dependencies between them. The next review Task depends on all three.
- Each review: a Task, or a DecisionTask where there are two outcomes (approved or not). Approve and reject are actions in the ActionRegistry.
- Rework: a failed or blocked review Task goes back to InProgress with start() (Failed → InProgress), or new tasks are added for the next round.
- New review rounds: AddWorkItemCommand and AddDependencyCommand add tasks with forward-only dependencies, so the graph never forms a cycle.
- Client escalation after 3 days: the account manager uses block() and assign actions with the executor set to the Account Director.

Design Patterns applied in the TIVIDY architecture

Composite

Problem in TIVIDY

- A workflow is a project made of tasks, and a task can have dependencies.
- Reports, JSON export, cloning and the list of available work must treat a single task and a whole project the same way.
- Without a common type, every operation would need special code for tasks and for projects.

Why Composite fits

- Composite lets clients treat one task and a whole project through the same interface.
- Project holds a list of WorkflowComponent children, so a project can hold tasks and decision tasks.
- Operations such as toJson(), getStatusSummary(), isCompleted(), collectWork() and clone() are written once on the interface and each class implements them.

Participants

- **Component:** WorkflowComponent (abstract)
- **Leaf:** Task, DecisionTask
- **Composite:** Project, with addWorkItem(), removeWorkItem(), getWorkItems() and findById()
- **Client:** ReportService, ProjectRegistry, the Commands

Collaboration

4. A Project is created and tasks are added with addWorkItem().
5. The Project owns its children. removeWorkItem() transfers ownership back to the caller.
6. A caller asks the Project for a status, a JSON export or the available work.
7. The Project calls the same operation on each child and combines the results.

Expected to vary

The kinds of work item and the way tasks are combined. The operations on the tree stay the same.

Example in our chosen scenario

The Agency Client Lifecycle is one Project. Its Tasks and DecisionTasks are its children.

Prototype

Problem in TIVIDY

- Users clone a project to reuse a good workflow, and carry a client's details into a new instance.
- Rebuilding the same tree by hand is error-prone, and a shallow copy would make two projects share the same tasks.

Why Prototype fits

- Each component copies itself through clone(), so the caller does not need to know the concrete class.
- Project::clone() deep-copies its children and remaps the dependency edges to the new copies.
- A clone resets the runtime data, so the copy starts fresh and does not affect the original.

Participants

- **Prototype:** WorkflowComponent::clone()
- **ConcretePrototype:** Task, DecisionTask, Project (each with a protected copy constructor)
- **Client:** WorkflowFactory::duplicate(), WorkflowEngineFacade::cloneProject() and duplicateTask()

Collaboration

1. The user asks to clone a project.
2. ProjectRegistry finds it and WorkflowFactory::duplicate() calls clone().
3. The Project copies each child and remaps the edges between the copies.
4. The new Project is added to the registry as a new session.

Expected to vary

Which component types exist. Each defines how it copies itself.

Example in our chosen scenario

When a client returns, the previous project is cloned into a new instance.

Builder

Problem in TIVIDY

- Projects are created from JSON templates, and each task has many optional parts (executor, deadline, dependencies, custom fields, branches).
- A constructor with all of these is hard to use, and a half-built project must never be exposed.

Why Builder fits

- Builder separates how a component is assembled from what it is.
- The Director validates the template first and then builds, and a failure frees the partial product.
- A new component type needs only a new builder registered with the Director.

Participants

- **Director:** Director, with registerBuilder(), createNewTask(), createNewProject() and createFromTemplate()

- **Builder:** ComponentBuilder (abstract), with setBasicInfo(), setDescription(), setExecutor(), setDeadline(), addDependency(), addCustomField(), addWorkItem() and build()
- **ConcreteBuilder:** TaskBuilder, DecisionTaskBuilder, ProjectBuilder
- **Product:** Task, DecisionTask, Project
- **Client:** WorkflowFactory

Collaboration

1. startInstance() asks WorkflowFactory to instantiate a definition.
2. The Director validates the template and picks the builder for each item type.
3. The Director creates fresh ids and wires the dependencies and branches by name in a second pass.
4. For decision tasks the Director looks up the router in the RouterRegistry.
5. build() returns the finished Project. If anything fails, the builder is reset and the partial project is freed.

Expected to vary

The contents of a template and the types of item it contains.

Example in our chosen scenario

The agency definition JSON becomes a Project with its Tasks and DecisionTasks.

State

Problem in TIVIDY

- A task is Pending, InProgress, Blocked, Failed, Complete, Cancelled or Skipped, and what it can do depends on that state.
- A task cannot be completed before it starts, and a cancelled task cannot be restarted.
- A switch statement in every operation would have to change whenever a state is added.

Why State fits

- Each state is its own class and handles only the events that are valid for it.
- Illegal events throw an InvalidTransitionError.
- A new state means a new class and no change to Task.

Participants

- **Context:** Task, DecisionTask
- **State:** TaskState (abstract), with start(), complete(), block(), fail(), cancel(), skip(), toString() and isSatisfied()
- **ConcreteState:** PendingState, InProgressState, BlockedState, FailedState, CompleteState, CancelledState, SkippedState

Collaboration

1. A new Task starts in PendingState.
2. A caller (a Command) calls start(), complete() or another event on the Task.
3. The Task passes the event to its current TaskState.
4. If the event is not allowed, the state throws an InvalidTransitionError and the task is unchanged.
5. Otherwise the state calls setState() on the Task, and the Task notifies its dependents.

Transitions

- Pending → InProgress, Blocked, Cancelled or Skipped

- InProgress → Complete, Blocked, Failed or Cancelled
- Blocked → InProgress, Failed or Cancelled
- Failed → InProgress or Cancelled
- Complete → InProgress
- Cancelled and Skipped are terminal.

Expected to vary

The states and transitions. The Task class does not change.

Example in our chosen scenario

If the client rejects the proposal, the review task fails, and the account manager restarts it after the rework.

Flyweight

Problem in TIVIDY

- Large projects hold many tasks, and each task has a state.
- If every task created its own state object, the same seven objects would be duplicated thousands of times.

Why Flyweight fits

- A state object holds no data about any one task. It only holds rules, and the task is passed in as an argument.
- One shared instance per state can therefore serve every task.

Participants

- **Flyweight:** TaskState
- **ConcreteFlyweight:** PendingState, InProgressState, BlockedState, FailedState, CompleteState, CancelledState, SkippedState (private constructors, a static instance())
- **FlyweightFactory:** TaskStateFactory::get(name)
- **Client:** Task (holds a non-owning TaskState* currentState)

Collaboration

1. A Task asks for or receives a shared state instance.
2. The state receives the Task as an argument when an event happens, so the Task's own data is the extrinsic state.
3. Commands and mementos store a state as a pointer, and nothing ever deletes a state.

Expected to vary

The number of tasks. The number of state objects stays at seven.

Example in our chosen scenario

Every task in every agency project shares the same InProgressState object.

Command

Problem in TIVIDY

- Users start, complete, fail or block tasks, add tasks and dependencies, edit information and route decisions.
- Every change must be recorded and undoable. If the engine called these directly it would be coupled to every operation.

Why Command fits

- Each operation becomes an object with execute(), undo() and describe().
- Undo and redo come from one history per project.
- Industries add actions through the ActionRegistry without changing the engine.

Participants

- **Command:** Command (abstract)
- **ConcreteCommand:** TaskStateCommand, AddWorkItemCommand, AddDependencyCommand, EditTaskInfoCommand, DecideCommand, RemoveWorkItemCommand
- **Receiver:** Task, DecisionTask, Project
- **Invoker:** CommandHistory, with run(), undo(), redo(), canUndo() and canRedo()
- **Client:** WorkflowEngineFacade, with ActionRegistry::create() building the command
- **Supporting:** ProjectSession (one per project), AuditLog, AuditEntry

Collaboration

1. The user requests an action. The Facade asks the ActionRegistry to build the Command.
2. ProjectSession::executeCommand() passes it to CommandHistory.
3. CommandHistory calls execute().
4. On success, the command is pushed onto the undo stack and the redo stack is cleared.
5. On failure, the command is recorded as REJECTED and deleted.
6. undo() takes the last command, calls undo() and moves it to the redo stack. redo() moves it back.

Expected to vary

The operations available. A new operation means a new command class and nothing else changes.

Example in our chosen scenario

"Approve discount" is a command registered by the industry. If the manager made a mistake, undo restores the previous state.

Memento

Problem in TIVIDY

- Editing a task's information changes its name, description, executor, deadline and custom fields.
- Undo must restore these without the history knowing how the Task stores them.

Why Memento fits

- The Task creates a snapshot of itself and later restores from it, so its private data stays private.
- TaskMemento has a private constructor and only Task can create one.

Participants

- **Originator:** Task, with createMemento() and restoreMemento()
- **Memento:** TaskMemento (name, description, executorId, deadline, participants, customFields)
- **Caretaker:** EditTaskInfoCommand, which holds the old and the new snapshot

Collaboration

1. Before applying an edit, EditTaskInfoCommand asks the Task for a memento.
2. It applies the new values and takes a second memento.
3. undo() restores the old one. redo() restores the new one.
4. The caretaker never looks inside the memento.

Expected to vary

The fields in a snapshot. Only Task and TaskMemento change.

Example in our chosen scenario

Changing a task's deadline can be undone and redone.

Note: State changes are undone by the command itself, which remembers the previous state as a pointer to a shared TaskState. A snapshot of a state would be pointless.

Observer

Problem in TIVIDY

- A task is ready only when everything it depends on is done.
- When a task finishes, fails or is undone, every dependent must react, and the task must not know who they are.

Why Observer fits

- A dependent registers itself with each of its prerequisites.
- When a prerequisite changes state, it notifies its dependents, and each one recomputes its readiness.

Participants

- **Subject:** WorkflowComponent, with addDependent(), removeDependent() and notifyDependents()
- **Observer:** WorkflowComponent::update(changedComponent)
- **Concrete:** Task and DecisionTask recompute readiness. Project re-evaluates its children.
- **Raised by:** setState()

Collaboration

1. addDependency() links the dependent and the prerequisite, and rejects a cycle with a CyclicDependencyError.
2. A prerequisite changes state through setState().
3. notifyDependents() calls update() on each dependent.
4. Each dependent recomputes whether it is ready.
5. If a prerequisite is Failed, active dependents become Failed. When the failure clears, they go back to Pending.

Expected to vary

The dependencies between tasks.

Example in our chosen scenario

When copy, visuals and media plan are all Complete, the internal review becomes ready.

Strategy

Problem in TIVIDY

- A decision task must choose a branch, for example whether a claim goes to approval or to investigation, or whether a discount is large.
- The rule differs between industries and processes, and must not live inside the task.

Why Strategy fits

- Each rule is its own class with one method, route(task, context).
- A DecisionTask holds a RoutingStrategy and calls it without knowing the rule.
- A new rule is a new class registered under a key.

Participants

- **Context:** DecisionTask, with setRouter(), addBranchTarget() and decide(context)
- **Strategy:** RoutingStrategy (abstract), with route(task, context) : string
- **ConcreteStrategy:** FieldValueRoutingStrategy, and the industry strategies
- **Registry:** RouterRegistry (owns the strategies, add/get/has/keys)
- **Client:** DecideCommand

Collaboration

1. The Director sets the router on the DecisionTask by key.
2. The user calls decide() with a JSON context.
3. The task calls route() and gets a branch label.
4. DecideCommand marks the selected branch and skips the others.
5. undo() restores the previous states and the previous selection.

Expected to vary

The routing rule and the branch labels.

Example in our chosen scenario

"Discount required?" routes on a field in the context.

Facade

Problem in TIVIDY

- The TextUI and the GUI need project creation, task actions, undo and redo, reports and history.
- These live in many subsystems (registries, factory, reports, repositories). If the UI called each directly, it would depend on all of them.

Why Facade fits

- One class gives the UI a simple entry point to the subsystems.
- It is not a god-object. It has no state of its own, only references to the subsystems, and it keeps no undo stacks.
- It implements three small interfaces, so each client depends only on what it uses.

Participants

- **Facade:** WorkflowEngineFacade
- **Interfaces:** ProjectManagement, Execution, Reporting
- **Subsystems:** ProjectRegistry, WorkflowFactory, ReportService, DefinitionRepository, ActionRegistry, RouterRegistry
- **Clients:** TextUI and the GUI

Collaboration

1. The UI calls an interface method, for example performAction(projectId, taskId, action).
2. The Facade finds the project and builds the command through the subsystems.
3. The Facade returns a result or a WorkflowError.
4. installModule(IndustryModule&) registers an industry's definitions, actions and routing rules.

Expected to vary

The UI and the industries. The subsystems stay unchanged.

Example in our chosen scenario

The TextUI calls startInstance("agency-client", "Kinetic Media – Client A") and nothing else.

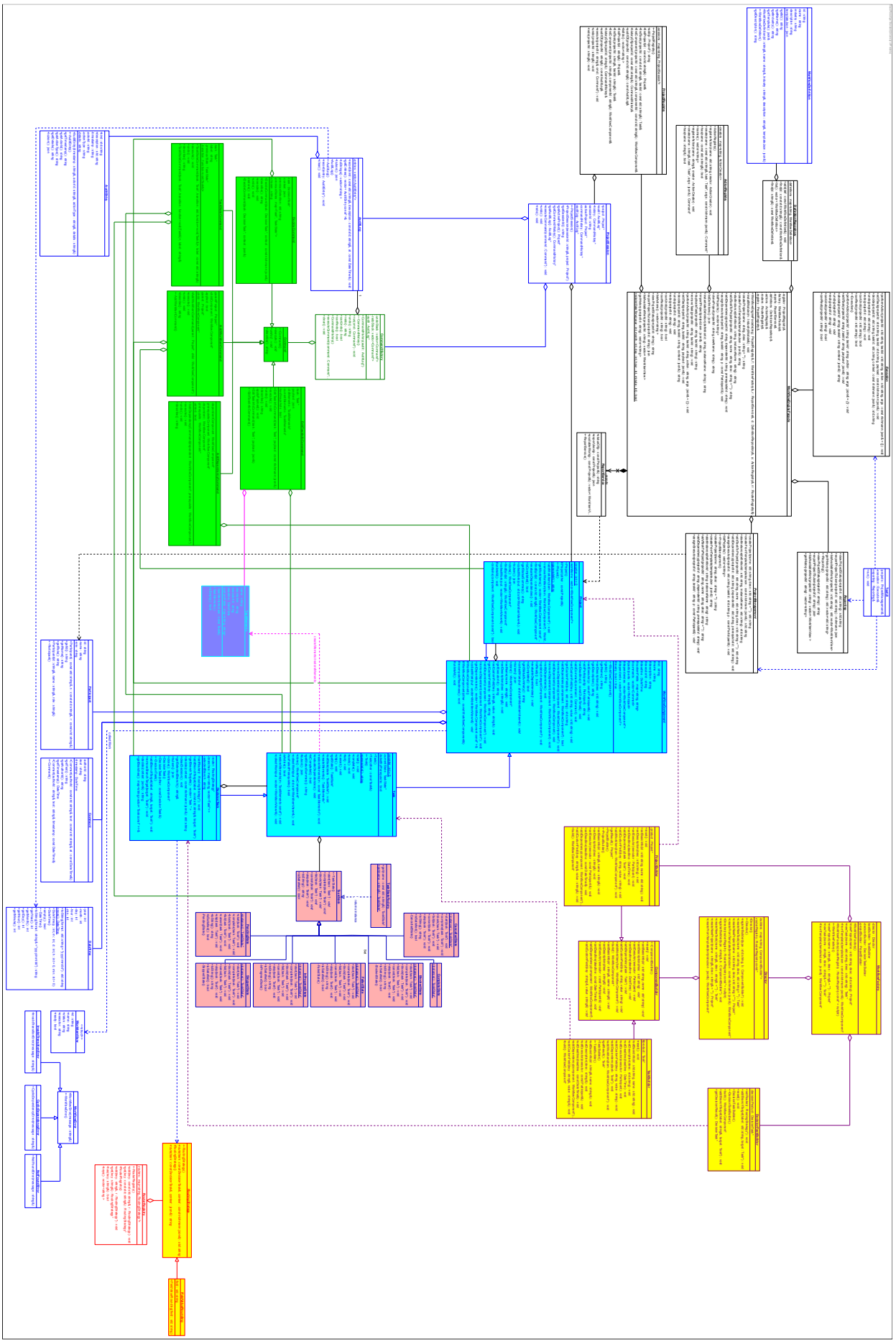
Runtime Behaviour Diagrams: State Diagram (Task Lifecycle)

The diagram shows a Task's lifecycle in TIVIDY. It applies to every task, including a DecisionTask. The states match the TaskState classes, and the transitions are the events on the task.

A new task starts in **Pending**.

- **start() [isReady()]:** Pending → InProgress, only when the task's prerequisites are satisfied.
- **block():** Pending or InProgress → Blocked.
- **start():** Blocked → InProgress.
- **complete():** InProgress → Complete.
- **fail():** InProgress or Blocked → Failed.
- **start():** Failed → InProgress, so a failed task can be retried.
- **start():** Complete → InProgress, so a completed task can be reopened.
- **cancel():** Pending, InProgress, Blocked or Failed → Cancelled.
- **skip() [non-selected branch]:** Pending → Skipped, when a DecisionTask selects a different branch.

Cancelled and Skipped are terminal. A Complete or Skipped task is *satisfied*, which means its dependents can start. Any event that is not listed throws an InvalidTransitionError, so the Task needs no large if/else statements.



Runtime Behaviour Diagrams

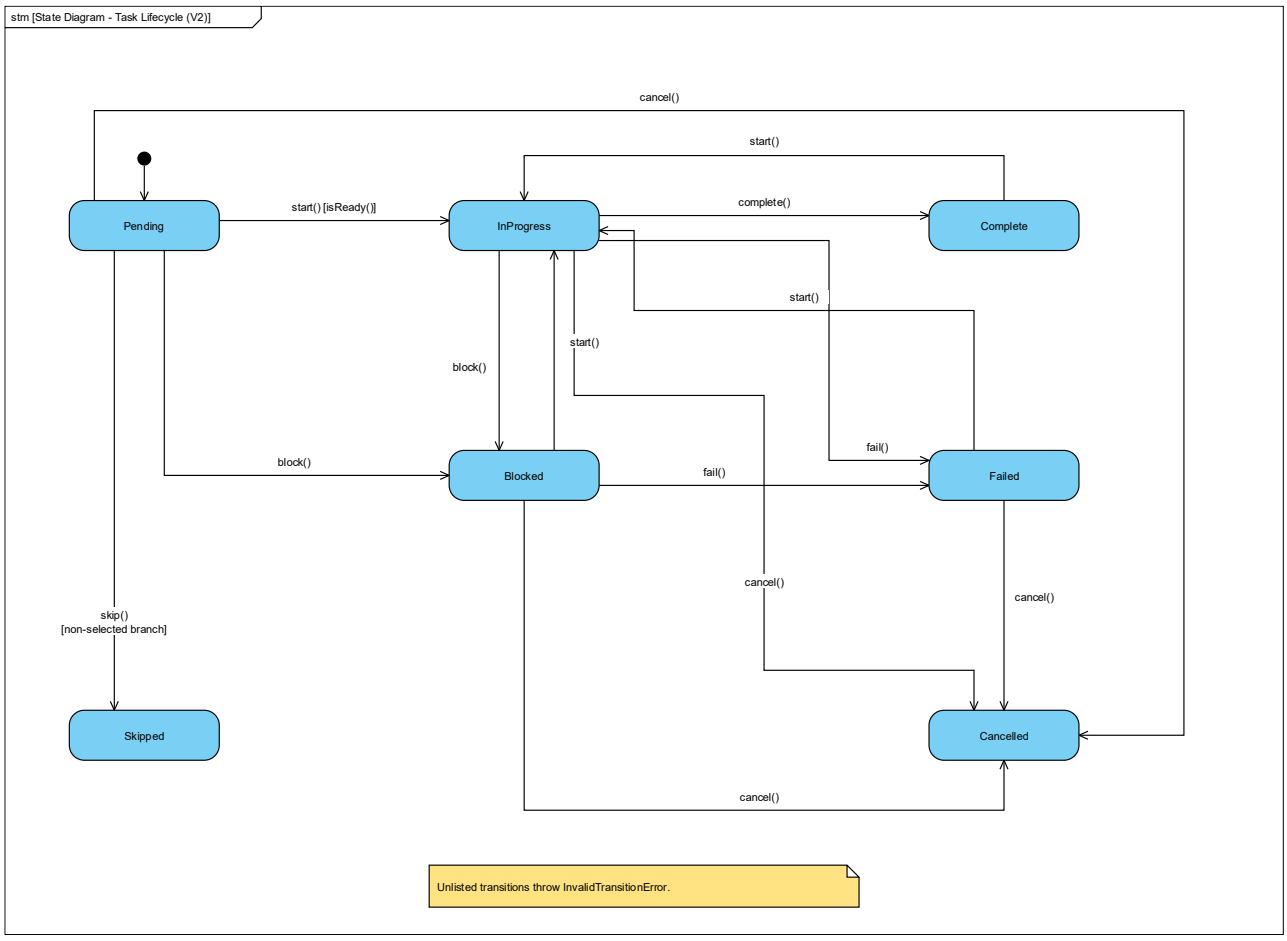
Runtime Behaviour Diagrams: State Diagram (Task Lifecycle)

The diagram shows a Task's lifecycle in TIVIDY. It applies to every task, including a DecisionTask. The states match the TaskState classes, and the transitions are the events on the task.

A new task starts in **Pending**.

- **start() [isReady()]:** Pending → InProgress, only when the task's prerequisites are satisfied.
- **block():** Pending or InProgress → Blocked.
- **start():** Blocked → InProgress.
- **complete():** InProgress → Complete.
- **fail():** InProgress or Blocked → Failed.
- **start():** Failed → InProgress, so a failed task can be retried.
- **start():** Complete → InProgress, so a completed task can be reopened.
- **cancel():** Pending, InProgress, Blocked or Failed → Cancelled.
- **skip() [non-selected branch]:** Pending → Skipped, when a DecisionTask selects a different branch.

Cancelled and Skipped are terminal. A Complete or Skipped task is *satisfied*, which means its dependents can start. Any event that is not listed throws an InvalidTransitionError, so the Task needs no large if/else statements.



Design Decisions and Revisions

Revision 1: From the whiteboard plan to a generic workflow engine

Problem: Our first plan was a fixed ad-agency flow: acquire the client, move them to Level 2, then Level 3, and mark them as Deleted if they lost interest. We realised this tied the whole system to one scenario, while TIVIDY is meant to be a generic workflow management system where the ad agency is only the example we run on it.

Original design: Client levels were fixed steps in the flow, and "Deleted" was the final state. Packages were built as a base package with add-ons, which suggested the Decorator pattern. Our initial pattern list had 11 patterns, including Factory, Mediator and Decorator.

Change: We turned the client levels into stages of a workflow template, so the engine no longer knows anything about ad agencies. We replaced "Deleted" with "Closed/Archived" so that the history is kept. Packages are now data stored on each running instance instead of classes. We also added win-back, which restores a client's old account from a saved snapshot. Finally, we settled on 12 patterns: Composite, Prototype, State, Template Method, Strategy, Chain of Responsibility, Observer, Command, Memento, Adapter, Visitor and Facade.

Why: The engine must work for any workflow, so it cannot depend on the ad agency scenario. Keeping closed clients as archived records preserves the audit history and makes undo, reports and win-back possible. Storing packages as data is simpler than creating a new class for every package or add-on. Win-back only works if the old account still exists to be restored.

Revision 2: From a fixed engine design to the implemented design

Problem: When we built the engine, the first design was larger than it needed to be. It had three task types, approval chains, task decorators, external-system adapters and a visitor over the tree. Several of these patterns had no clear problem to solve in the code.

Original design: 12 patterns. Tasks were ManualTask, ApprovalTask and ServiceTask with a Template Method for execute(). Approvals used a Chain of Responsibility. Extra behaviour used Decorator. External systems used Adapter, and reports used Visitor.

Change:

- **Template Method, Decorator:** removed. There is now one Task, and a DecisionTask for branching. The behaviour that differed by task type is now in the State classes and in the Commands.
- **Chain of Responsibility:** removed. Approval is a DecisionTask routed by a Strategy, with approve and reject registered as actions.
- **Adapter:** removed. TIVIDY manages the process only, and outcomes of external systems are recorded on tasks by people.
- **Visitor:** removed. Reports use the Composite's own operations (collectWork, toJson, getStatusSummary), read by ReportService.
- **Flyweight:** added. The task states are shared objects.
- **Builder:** added. A Director and builders create projects and tasks from JSON templates.
- **Pattern set:** Composite, Prototype, Builder, State, Flyweight, Command, Memento, Observer, Strategy and Facade (10 patterns).
- **Observer:** now works through task dependencies and no longer through an event bus.

- **Memento:** now backs the undo of edits to a task's information, and no longer backs a win-back snapshot.
- **Facade:** now sits behind three small interfaces, so no class is a god-object.
- **Ownership:** raw pointers with clear ownership (a Project owns its tasks, CommandHistory owns its commands, and states are shared and never deleted).

Why: Each pattern that remains solves a problem we can point to in the code. The patterns we removed had no corresponding code, and keeping them would make the document differ from the implementation. The smaller design is easier to extend: an industry adds definitions, actions and routing rules through an IndustryModule without changing the engine.

GitHub Workflow

Link to GitHub repository: https://github.com/MogaleLebethe/COS214-Project-2026

We worked on separate branches then pushed to the dev branch. Not pushes were allowed on the main branch without being pushed to dev. Every pull request to dev had to have at least one approval.

Bibliography

[1] H. A. Reijers, I. Vanderfeesten, and W. M. P. Van Der Aalst, “The effectiveness of workflow management systems: A longitudinal study,” *Int. J. Inf. Manage.*, vol. 36, no. 1, pp. 126–141, Feb. 2016, doi: 10.1016/J.IJINFOMGT.2015.08.003.

[2] W. M. P. van der Aalst and K. van Hee, *Workflow Management: Models, Methods and Systems*. The MIT Press, 2002. doi: 10.7551/mitpress/7301.001.0001.

[3] C. Ouyang, M. Adams, M. T. Wynn, and A. H. M. ter Hofstede, “Workflow Management,” in *Handbook on Business Process Management 1*, Second Edition., J. vom Brocke and M. Rosemann, Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 2015, pp. 475–506. doi: 10.1007/978-3-642-45100-3_21.

[4] G. Kappel, S. Rausch-Schott, and W. Retschitzegger, “A framework for workflow management systems based on objects, rules and roles,” *ACM Comput. Surv.*, vol. 32, no. 1es, p. 27, Mar. 2000, doi: 10.1145/351936.351963.

[5] D. Georgakopoulos, M. Hornick, and A. Sheth, “An overview of workflow management: From process modeling to workflow automation infrastructure,” *Distrib. Parallel Databases*, vol. 3, no. 2, pp. 119–153, Apr. 1995, doi: 10.1007/BF01277643.